

Bot Discord de recommandation de recettes

RAG – PostgreSQL/pgvector – LLM local – Discord.py

Elazar COHEN

Soutenance L3 Informatique

Plan de la présentation

- 1 Introduction
- 2 Architecture technique
- 3 Problème de performance
- 4 Organisation du travail
- 5 Bilan et améliorations

L'idée de départ

Aider les gens à cuisiner **en fonction de leur budget**.

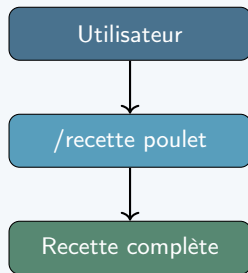
Évolution du projet

Application mobile → hors compétences fac, déjà réalisé



Bot de groupe Discord : communauté, réponse instantanée

Choix de Discord : facile à botter, expérience naturelle de groupe



Choix techniques

Base de données

PostgreSQL avec extension
pgvector pour la recherche
vectorielle

LLM local

Mistral via Ollama. Pas d'API
payante. Contrôle total sur le
système

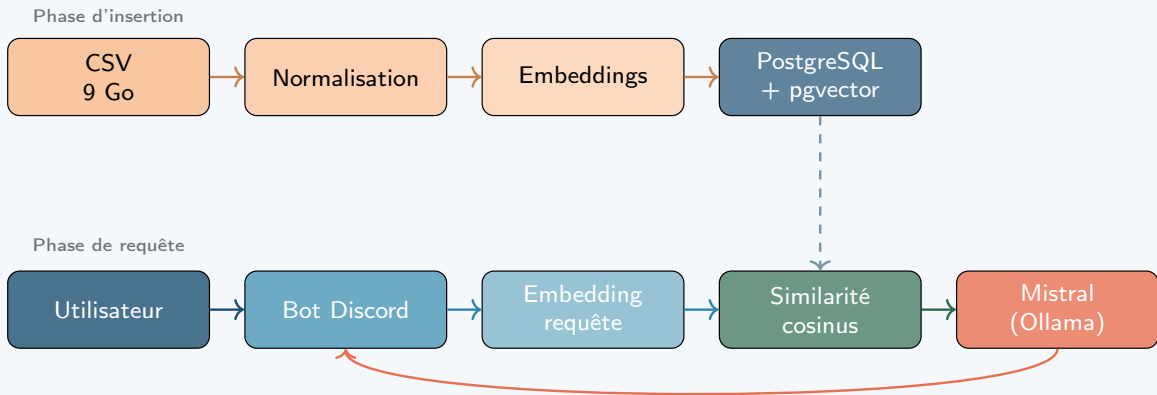
Bot Discord

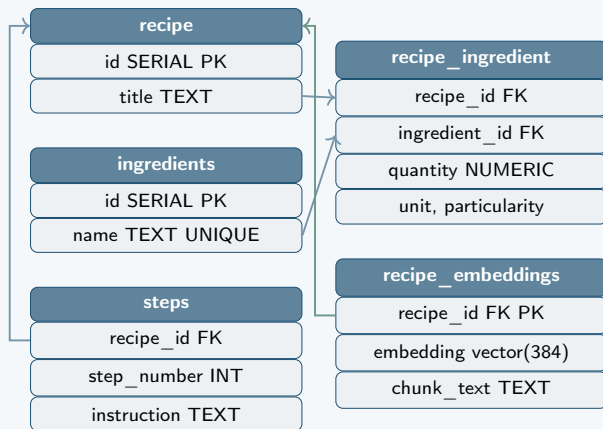
discord.py. Commande /recette.
Réponse async structurée

Technique clé : RAG — Retrieval-Augmented Generation

Recherche contextuelle + Génération LLM

Architecture globale — pipeline complet





Points clés :

- 5 tables
- Ingrédients dédupliqués (UNIQUE)
- vector(384) = all-MiniLM-L6-v2
- chunk_text = texte complet pour le LLM

Requête de recherche

```
SELECT recipe_id,  
       1 - (embedding <=> query)  
FROM recipe_embeddings  
ORDER BY embedding <=> query  
  
LIMIT 5;
```

2h30

pour 10 000 recettes

Dataset complet : $\approx$ 2 millions de recettes. Estimation : plusieurs semaines

Code initial :

- 1 connexion ouverte par recette
- 1 INSERT par ligne, par table
- Embedding calculé recette par recette
- Des **dizaines de milliers** d'aller-retours avec la BDD

Problème (pseudo-code)

```
for recipe in dataset:
    emb = model.encode(recipe)
    conn = db.connect()

    cur.execute("INSERT ...")
```

Deux goulots d'étranglement identifiés

Goulot #1 — Transactions

Ouvrir une connexion + transaction pour chaque recette = overhead énorme.

PostgreSQL doit valider (COMMIT) à chaque fois.
10 000 recettes x 5 tables = **50 000+ requêtes**.

Solution : une seule transaction globale + COPY PostgreSQL

Goulot #2 — Embeddings (90–95% du temps)

`model.encode()` sur une recette à la fois = très inefficace.

SentenceTransformer est optimisé pour les **lots (batches)**. 1 à la fois = pas de vectorisation CPU.

Solution : `model.encode(chunks, batch_size=1024)`

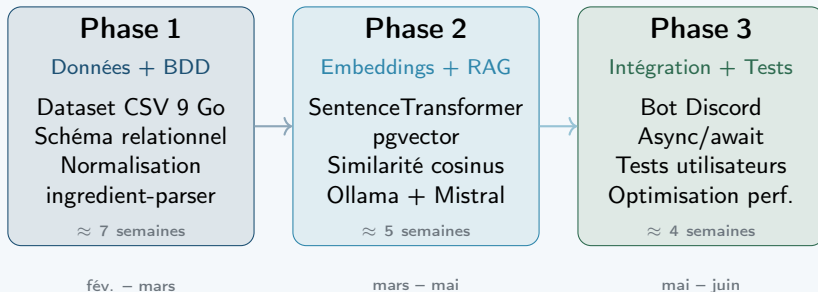
Résultats — comparaison des performances

Etape	Temps	Gain	Changement
Etat initial	2h30	—	INSERT un par un, encode un par un
Batch embeddings	≈10 min	×15	batch_size=1024
+ COPY PostgreSQL	3–5 min	×50	CSV en memoire, 1 transaction
Insertion BDD seule	<1 sec	—	Quelques millisecondes
Embeddings seuls	3–5 min	—	90–95% du temps total

Limite restante

Les embeddings sur **CPU** restent le vrai goulot. Sur GPU, le gain serait de x10 a x50 supplémentaires — mais les machines de la fac n'ont pas pgvector installé.

Organisation du travail — 3 grandes phases



Défi de la normalisation

Données en anglais, unités américaines (cups, oz).
Utilisation de `ingredient-parser` pour extraire
nom, quantité, unité.

Tests utilisateurs

Bot lancé dans un serveur Discord privé. Retours sur
la qualité des recettes. Amélioration du prompt
système.

Ce qui fonctionne

- Bot utilisable en conditions réelles
- RAG efficace sur des requêtes vagues
- 10 000 recettes insérées en 3–5 min
- Pipeline complet autonome (LLM local)
- Prompt structuré : recette cohérente

Limites identifiées

- Mistral local moins puissant qu'un LLM cloud
- Qualité dépendante du dataset
- Embeddings sur CPU : goulot restant
- Pas de filtre allergies / régime

Performance

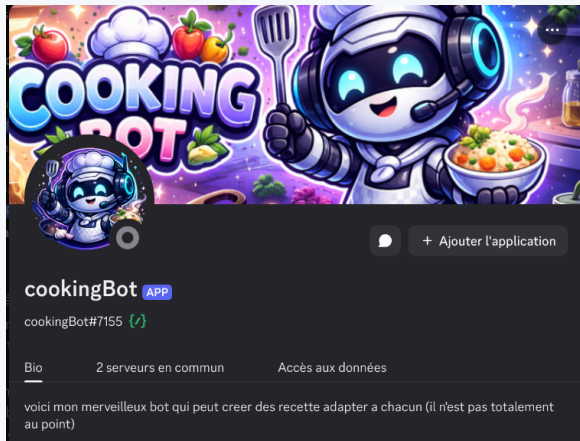
1. **Embeddings sur GPU** : x10 a x50 plus rapide, dataset complet en quelques minutes

Fonctionnalités

2. **Filtrage nutritionnel** : recherche vectorielle + filtres SQL (budget, allergies, régime)
3. **LLM plus puissant** : API cloud pour la génération, recherche vectorielle en local

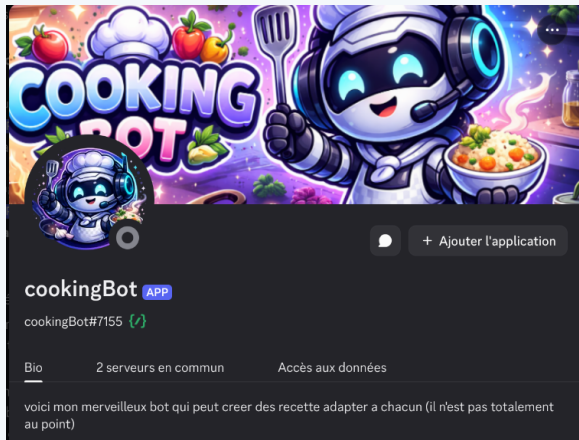
Ce projet est extensible : BDD, embeddings et LLM sont des briques indépendantes.

Exemple d'utilisation du bot

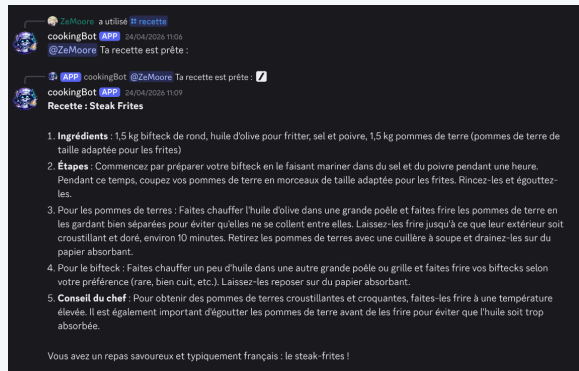


Profil Discord du bot

Exemple d'utilisation du bot



Profil Discord du bot



Exemple de recette générée via /recette

Ce que ce projet m'a appris :

- Construire un système complet de bout en bout (données → BDD → LLM → bot)
- Comprendre et implémenter le **RAG**
- Diagnostiquer des problèmes de **performance réels** (x50 de gain)
- Travailler avec des bibliothèques NLP (SentenceTransformer, ingredient-parser)
- Développement async avec discord.py

Stack technique

PostgreSQL + pgvector
SentenceTransformer
all-MiniLM-L6-v2
Ollama + Mistral
discord.py
Python 3.11

Merci pour votre attention

Questions ?